

Model Selection One-Pager

The four levers, the five model classes, and the back-of-the-napkin decision flow. Pin it up. Names rotate; classes are stable.

THE FOUR LEVERS

1 · Parameter count vs effective intelligence

Bigger is not always better. A 30B reasoning-tuned model often beats a 70B chat-tuned model on structured output and code. Reasoning training matters more than raw size at the margins that affect your work.

3 · Quantization vs quality

Q4 is fine for chat. Q8 or fp16 for code, structured output, or anything where one wrong token tanks the result. On hosted APIs this is the provider's problem; it becomes your lever when you run open models yourself.

2 · Context window vs cost

You pay per token in and out. Frontier models are genuinely good at finding one fact in 200K tokens; reasoning and synthesis across the whole window fall off well before the advertised limit. Measure on your own task.

4 · Latency vs throughput

Streaming chat needs first-token latency under 500ms or it feels broken. Batch jobs care about tokens-per-second-per-dollar instead. Different model classes win in each regime.

THE FIVE MODEL CLASSES (2026)

CLASS	EXAMPLES (VERIFY CURRENCY)	WHEN TO USE
Frontier reasoning	Claude Opus / Sonnet, GPT-5.5, Gemini 3 Pro	High-stakes structured output. When wrong-answer cost > API fee.
Mid-size open	Llama 4, Qwen3 72B-class, Mistral Large	Synthesis, long context, balanced cost.
Code-specialized	Qwen3-Coder, Codestral, GLM	Code generation, refactoring, structured output that is mostly code.
Small efficient	Llama 4 small variants, Qwen3 8B, Phi-4, Gemma 3	Classification, routing, simple chat. Latency-sensitive.
Multimodal	GPT-5.5, Gemini 3 Pro; Claude Opus 4.8 / Sonnet 5 (images + PDFs)	Image, audio, or video input. Audio or video input still means GPT or Gemini; Claude takes images and PDFs.

THE DECISION FLOW (BACK OF THE NAPKIN)

Is the input an image, audio, or video? → **Multimodal**. Audio/video narrows the field further; every frontier model reads images.

Is the output mostly code? → **Code-specialized** (or frontier reasoning when a subtle bug is expensive).

Is it classification, routing, or simple chat where latency matters? → **Small efficient**. Cheapest and fastest; wrong answers are cheap to catch.

Does a wrong answer cost more than the API call? → **Frontier reasoning**. Pay for the thinking when the stakes justify it.

Everything else (synthesis, long context, balanced cost) → **Mid-size open**. The workhorse tier.

Two standing rules. Model names go stale within 6 months; the classes are what you memorize. And benchmark scores are gamed and often misleading: your eval set, on your data, decides which specific model wins inside a class.